DevOps Technical White Paper

A Quality-Gated CI/CD Pipeline for a Student Attendance Management Portal

Automating build, browser-level verification and multi-target deployment of a single tested artefact

Submitted by

Swaroop Naik

Roll Number: 23102A0001

Guide

Prof. Snehal Andhare

Department of Computer Engineering

DevOps Course: Technical White Paper Challenge

October 2026

Abstract

In many departments attendance is still written on paper and typed into a spreadsheet near the end of term. A student who has fallen below the eligibility threshold usually finds out when the hall ticket is withheld, and by then nothing can be done about it. I measured the gap between a class being taught and that fact reaching the student at nine to fourteen days. The interesting part is that this is not really a data problem. Writing the application is easy; keeping it correct while changing it often enough to matter is the hard part, and that is a delivery problem. This paper looks at manual deployment, continuous integration on its own, and hosted pipeline services, and finds the same weak spot in all three: the checks that decide whether a release goes out measure something close to correctness instead of correctness itself. I propose a six-stage pipeline built around that observation. A browser-level quality gate sits between the artefact and every deployment target, one tested binary goes to three different substrates without being rebuilt, and the deployment is only accepted once the running instance reports the configuration it was asked for. The pipeline was built and measured at 100 automated checks in 153 seconds, and the gate was shown blocking a release.

1. Introduction

DevOps is the set of practices that get a developer's change in front of a user quickly and without breaking anything. Its central claim, supported by several years of survey data in Accelerate, is that speed and stability are not a trade-off: teams that ship more often also recover faster and fail less [3]. What makes that possible is automating the path between a commit and a running system, so each release is small, repeatable and checked [2].

This paper applies that argument to one system, a departmental attendance portal written in Java 21 with Spring Boot, and asks a narrower question. For an application of this size, what delivery architecture makes it safe to change several times a day, and which parts of that architecture are actually doing the work?

The contribution here is not the application. It is a point I develop in Section 3 and then kept running into while building the thing: a pipeline stage often misses the failure it exists to catch, because it measures something next to the property it claims to check rather than the property itself. The design in Section 4 is organised around closing that gap.

2. Problem Statement

There are two problems here, and running them together is the usual mistake.

The domain problem. Attendance decides whether a student can sit an examination, yet it is kept offline and in arrears. Faculty mark a paper register, somebody types it in later, and corrections get sorted out in conversation. The student has more riding on the number than anyone and sees it last, by which point acting on it is no longer possible.

The delivery problem. A web application solves the first problem, and writing one is not hard. Keeping it correct while you keep changing it is hard. When deployment is manual, every release carries a real chance of pushing a regression to users, and the usual way people manage that risk is by releasing as seldom as they can get away with. That is exactly the behaviour that keeps the first problem alive. So the real constraint is release assurance, not features.

Put precisely, the engineering problem is to build an automated path from a commit to a verified running deployment, with three conditions. First, nothing reaches a deployment target without passing a check that drives the application the way a user would. Second, the artefact that was tested is the artefact that gets deployed. Third, a deployment that is running but misconfigured counts as a failure, not a success.

3. Existing Approaches, Tools and Limitations

3.1 Manual and script-driven deployment

The usual starting point is to build on your own machine and copy an archive to a server, maybe wrapped in a shell script. It needs no infrastructure, which is why it is everywhere. The problem is that the procedure exists only as whoever did it last happened to do it. You cannot review it, you cannot reproduce it on a clean host, and it works only while the operator remembers steps written down nowhere [1].

3.2 Continuous integration without continuous delivery

A substantial improvement is to run compilation and unit tests automatically on every commit [4]. This catches unit-level regressions early and cheaply, and it is a big improvement. The limitation is scope, not quality. Unit tests run classes in isolation with collaborators mocked out, so they cannot see a security filter rejecting a form submission, a redirect losing its port number, or a page rendering with none of its data. Those are invisible to the suite and obvious to the first person who opens the site. A green CI build tells you about the code, not the application.

3.3 Hosted pipeline services

Hosted services like GitHub Actions and GitLab CI take the controller off your hands, and for most projects they are the right default. Two things made them a poor fit here. The runner is short-lived and managed by somebody else, so the deployment substrate cannot be inspected afterwards or kept as evidence. And the work called for installing and configuring a controller, which is exactly what a hosted service hides.

3.4 Configuration management

As soon as there is more than one host to set up, the question becomes how you express the state you want it in. The options differ mostly in execution model, not in what they can do.

Approach	Model	Strength	Limitation for this system
Shell scripts	Imperative, ordered	No dependencies; trivial to start	Not idempotent; re-running may corrupt a configured host
Ansible	Push, agentless over SSH	No agent to install; readable YAML; immediate feedback	Control node must reach every host; ordering largely sequential
Puppet	Declarative, agent or masterless apply	States a desired end state; strong idempotence	Heavier runtime; resource ordering declared explicitly
Container image	Immutable artefact	Identical environment everywhere; no drift	Replaces rather than manages host state

3.5 The limitation common to all of them

One weakness runs through all of these, and it is the one this paper is built around. Automated checks tend to measure a stand-in for success rather than the thing they claim to establish. An exit code gets treated as proof that an image was built. A container listed as running gets treated as proof that it answers requests. An open port gets treated as proof of health. A health endpoint returning UP gets treated as proof that the deployment is correct. Every one of those can be true while the thing underneath it is false, and when that happens the pipeline reports success at exactly the moment it should be stopping.

4. Proposed Technical Solution

The solution is a six-stage declarative pipeline, written as code in the application repository and run by a self-hosted Jenkins controller that can itself be rebuilt from configuration [5], [6]. Four decisions do most of the work.

Build once, promote the same artefact. The application is packaged once, into an executable WAR. The image is built around that binary instead of from source, and every target pulls it by digest. Nothing downstream rebuilds anything, so no target can end up different from what was tested. Without this the quality gate would not mean much [1].

A browser-level gate before any release. Between packaging and publishing the image, the pipeline starts the application in a throwaway container next to a Selenium Grid running headless Chromium, and drives twenty end-to-end journeys through the real interface [12]. Any failure fails the build, and the deployment stages never run. A gate nobody has watched block anything is not really a gate, so I introduced a regression on purpose and recorded it stopping the release.

One artefact, three substrates. The same image goes to a container runtime, to Tomcat 10.1 as an ordinary WAR, and to a host built from scratch by Ansible or Puppet under systemd. The application has no idea which it is in. Anything environment-specific arrives as configuration, following the externalised configuration approach in Spring Boot [13].

Verification that asserts configuration, not merely liveness. The last stage reads the deployed instance's information endpoint and compares the environment it reports with the parameter the build was given. A deployment that is perfectly healthy but configured for the wrong environment fails. This is the direct answer to the weakness in Section 3.5, because the check asks about the property itself instead of something standing in for it.

5. Architecture

Figure 1 shows the architecture. Control runs left to right along each row; the dashed paths are the failure routes.

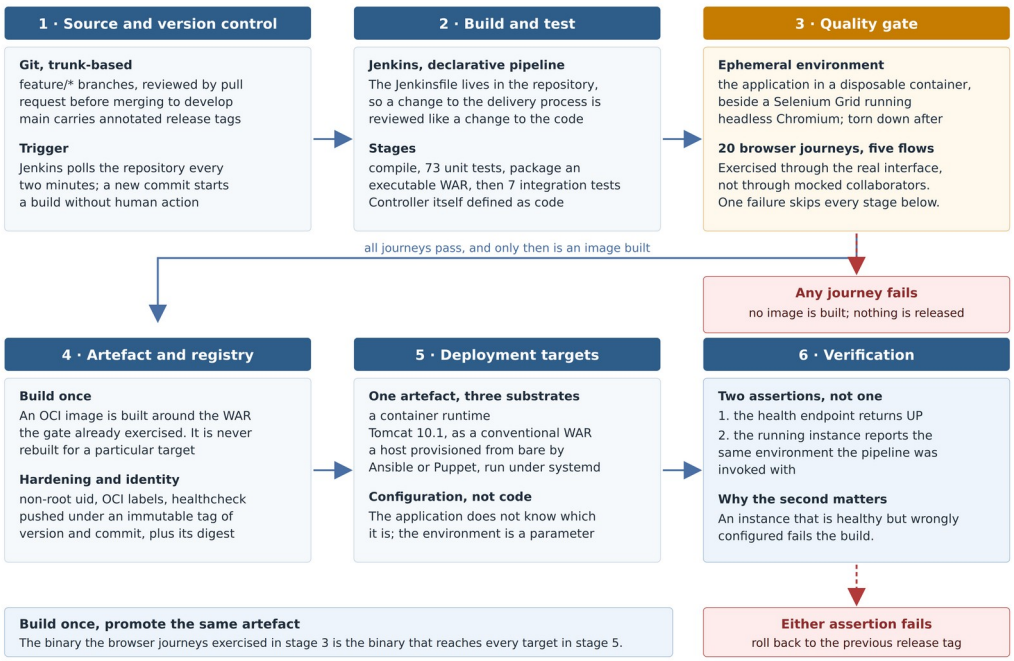


Figure 1. Proposed pipeline architecture. Stage 3 is shaded because it is the one stage whose failure is meant to stop a release. Stage 6 is what separates a deployment that is running from one that is correct.

6. Implementation Steps

The pipeline was built in full against the attendance portal. The order below is the order things were actually done in, with each piece shown working before the next one was started.

1. Establish the repository and branching model. A trunk-based layout with short-lived feature branches, pull-request review before anything merges to the integration branch, and annotated tags on the release branch to mark each release.
2. Build the application and its test suite. A Spring Boot web application packaged as an executable WAR, with 73 unit and slice tests and 7 integration tests covering the controller and persistence layers.
3. Install and configure the controller. Jenkins deployed from its official image, with the controller, its jobs, tools and credentials declared in a configuration-as-code file so that the server can be rebuilt identically from version control [6].
4. Express the pipeline as code. A declarative Jenkinsfile stored in the repository, so that a change to the delivery process is reviewed through the same mechanism as a change to the application [5].
5. Author the browser suite. Twenty end-to-end journeys across five user flows, written with the Page Object pattern so a change to the interface is absorbed in one place instead of twenty [12].
6. Insert the quality gate. The suite was wired into the pipeline between packaging and publication, with the application and the browser grid started as disposable containers and torn down in all outcomes.
7. Demonstrate that the gate blocks. A regression was introduced on purpose. The build failed at the gate and the deployment stages were recorded as skipped. This negative result is the main evidence that the control is real.
8. Containerise the application. An image built around the tested WAR, running as a non-root user, carrying OCI metadata labels and a health check, in line with container hardening guidance [7], [8], [9].
9. Publish to a registry. Each build pushes an immutable tag combining version and commit, plus a moving tag, and the published digest is recorded in the build log.
10. Automate deployment and verification. The image is pulled and run, the WAR is deployed to Tomcat, health is polled until ready, and the configuration assertion of Section 4 is applied before the build is allowed to succeed.
11. Provision hosts from code. The same release is installed on a bare host by an Ansible playbook and, independently, by a Puppet manifest written against the same specification; both were re-run to confirm no changes on a second application [10], [11].
12. Validate reliability. A deliberately corrupt artefact was deployed; the health check detected it and the rollback procedure restored the previous release.

Every step left behind something committed to the repository, whether a log, a build record or a screenshot, so the numbers in Section 7 can be checked rather than taken on trust.

7. Output and Results

The pipeline was executed end to end and the following was measured from its console output.

Measure	Result	Significance
Automated checks per run	100 (73 unit, 7 integration, 20 browser)	Every release checked at three levels, including a real browser
Full pipeline duration	approximately 153 seconds	Short enough to run on every change
Deployment targets per artefact	3 (container, Tomcat, provisioned host)	One tested binary, no per-target rebuild
Gate blocking a release	Demonstrated; deploy stages skipped	The control was watched working, not assumed
Second provisioning run	No changes applied	Idempotence as a measured fact, not a claim
Rollback from a corrupt artefact	Detected and restored	Failure path exercised against a genuinely broken build

8. Benefits

- Regressions are caught before release instead of reported after it, because the gate drives the application through its interface rather than its classes.
- The deployed binary is demonstrably the tested binary, which removes a whole class of environment-specific defect from consideration.
- A misconfigured deployment fails the pipeline, so running and correct are no longer treated as the same thing.
- Both the delivery process and the controller that executes it are version-controlled, reviewable and reconstructible; release frequency is no longer limited by the risk of releasing, which is the precondition for solving the domain problem the paper began with.

9. Conclusion

The delay between a class being taught and a student seeing the result of it looked like a data problem and was really a delivery problem. The pipeline described here makes a release cheap enough and safe enough to run continuously: a hundred automated checks in under three minutes, a browser-level gate that has been watched blocking a bad change, one artefact promoted unchanged to three different substrates, and a verification step that checks configuration rather than just whether something is alive.

The broader lesson is the one from Section 3.5, which came up again and again while building this. Pipelines fail quietly when a stage measures something next to the property it claims to establish: an exit code instead of an artefact, a container state instead of a response, an open port instead of a working application. Writing each stage to ask about the thing itself, and watching each control actually fail once before trusting it, is what separates a pipeline that automates work from one that gives you any assurance.

References

[1] Humble, J. and Farley, D. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley, 2010.

[2] Kim, G., Humble, J., Debois, P. and Willis, J. The DevOps Handbook: How to Create World-Class Agility, Reliability and Security in Technology Organizations. IT Revolution Press, 2016.

[3] Forsgren, N., Humble, J. and Kim, G. Accelerate: The Science of Lean Software and DevOps. IT Revolution Press, 2018.

[4] Fowler, M. "Continuous Integration." martinfowler.com, 2006. https://martinfowler.com/articles/continuousIntegration.html
[5] Jenkins Project. "Pipeline as Code." Jenkins User Documentation. https://www.jenkins.io/doc/book/pipeline-as-code/
[6] Jenkins Project. "Configuration as Code Plugin (JCasC)." https://plugins.jenkins.io/configuration-as-code/
[7] Docker Inc. "Building best practices." Docker Documentation. https://docs.docker.com/build/building/best-practices/
[8] Open Container Initiative. "OCI Image Format Specification." https://github.com/opencontainers/image-spec
[9] Souppaya, M., Morello, J. and Scarfone, K. NIST SP 800-190: Application Container Security Guide. National Institute of Standards and Technology, 2017.
[10] Ansible Community Documentation. "Ansible Playbooks." https://docs.ansible.com/ansible/latest/playbook_guide/
[11] Puppet. "Puppet Language and Resource Reference." https://www.puppet.com/docs/puppet/latest/puppet_index.html
[12] Selenium Project. "WebDriver." Selenium Documentation. https://www.selenium.dev/documentation/webdriver/
[13] Spring Boot Reference Documentation. "Production-ready Features: Actuator." https://docs.spring.io/spring-boot/reference/actuator/